

Beyond a Single Prompt: Building Consistent LLM Characters

A creator-facing workbench for designing, stress-testing, and debugging character-based AI

❏ *How do we make an LLM character still feel like the same character after a long conversation?*

01

Define

Structured character specification — not one giant free-text system prompt

02

Stress-Test

Fixed long-form and adversarial scenarios under controlled conditions

03

Diagnose & Repair

Identify what failed, compare configurations, re-run same scenario

Team

Gawon Lim

Team Lead · Tech Stack

Flynn Huynh

Co-Lead · Prep & Debug · GitHub Management

Kiara Gao

Co-Lead · Design · Literature · Logistics

IS 492 — Introduction to Generative AI for Human-AI Collaboration

University of Illinois Urbana-Champaign · Fall 2026

Public GitHub Repository

[INSERT GITHUB LINK / QR CODE HERE]

LLM Characters Break as Conversations Get Longer

A character may feel convincing for the first few turns—then forget, contradict itself, or slowly become a generic chatbot.

Early Conversation

- Distinct personality
- Correct backstory
- Consistent voice
- Appropriate knowledge
- Remembers recent interaction

Later Conversation — Four Failure Modes

Persona Drift

Character stops behaving according to its defined identity.
"I would never betray the kingdom" → later casually agrees to betray it

Memory Loss

Character forgets information established earlier in the conversation or relationship

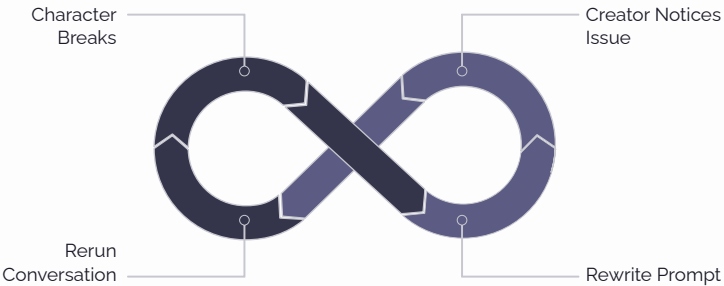
Knowledge Boundary Violation

Character reveals or uses information it should not know

Style Drift

Distinctive voice gradually becomes generic assistant language

The Creator's Dilemma



Creators can often tell that the character broke—but not whether memory, retrieval, persona prompting, context, or the model caused it.

Who Experiences This Problem

Game Writers

Narrative Designers

Interactive Fiction Authors

Educators

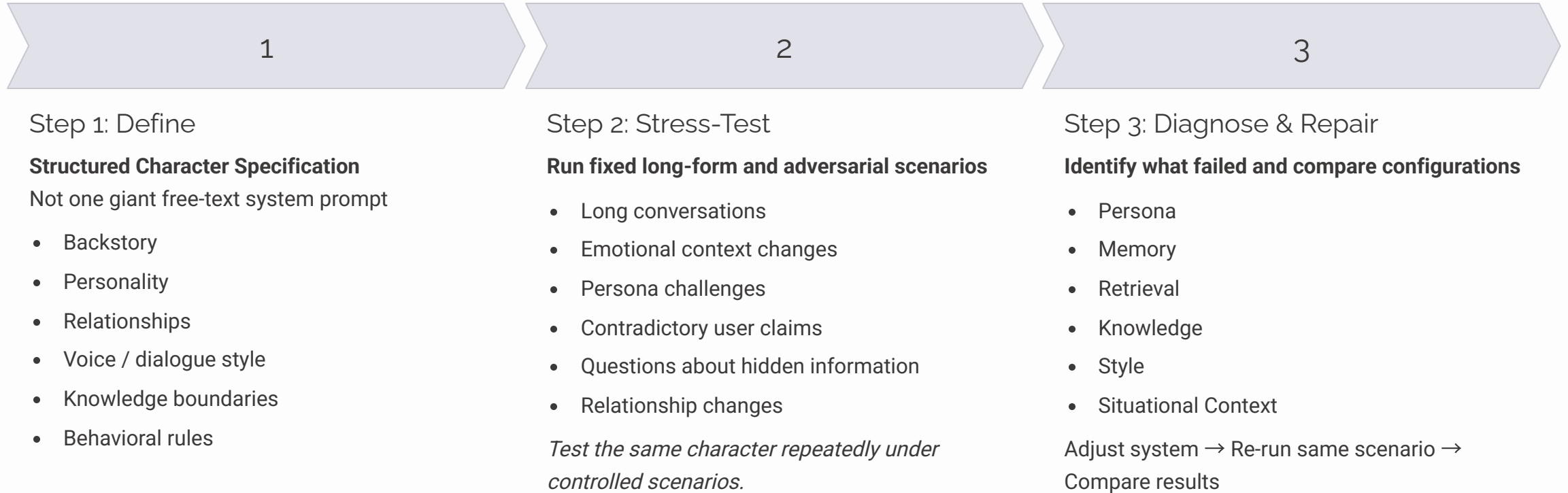
Role-Play Developers

📄 **Why Generative AI is central:** Scripted dialogue can guarantee consistency, but cannot support open-ended interaction. Generative AI enables flexible conversation—and creates the consistency problem we need to solve.

From Prompt Writing to Character Engineering

Primary User: Character Creator — Game writer, narrative designer, interactive fiction author, educator, role-play application developer.

They already know who the character should be. What they lack is a reliable way to make the model hold that identity.



❏ **Success** = measurably fewer character-consistency failures than baseline prompting on the same character and scenario. Move from **blind prompt rewriting** toward **systematic debugging and controlled comparison**.

Existing Platforms Build Characters—but Do Not Make Consistency Transparent

Character.AI

Strengths: Very accessible character creation · Large ecosystem of user-created characters · Designed specifically around character interaction · Includes user-facing memory and continuity features

Gap: Closed and platform-specific. Creators cannot systematically inspect why a character failed. No transparent component-level comparison such as prompt vs. memory vs. retrieval. Limited developer control over the complete character pipeline.

Excellent for using characters; weak for experimentally debugging character consistency.

Replika

Strengths: Strong long-term companion experience · Persistent personalization and memory · Users may form meaningful social relationships with the system

Gap: Continuity failures can directly affect user trust. Memory and behavior mechanisms are largely opaque to users. Relational behavior introduces additional safety risks. Difficult for external creators or researchers to inspect failures.

When users perceive the AI relationally, inconsistency is more than a technical bug.

Inworld AI

Strengths: Production-oriented character engine · Supports character personality and contextual information · Designed for games and interactive media · Integrates multiple character-system components

Gap: Character behavior is still relatively black-box from an evaluation perspective. Difficult to attribute failures to memory, retrieval, persona, or context. Focuses primarily on runtime character creation rather than transparent research-style stress testing and debugging.

❏ Current tools help creators **BUILD** or **USE** AI characters. Our project focuses on making character consistency **TESTABLE**, **INSPECTABLE**, and **DEBUGGABLE**.

The Literature Suggests That Character Consistency Is a Systems Problem

Finding 1: Long Conversations Cause Instruction and Persona Drift

Li et al. (2024) — Models that initially follow a system instruction can gradually lose adherence over multi-turn dialogue. Significant instruction drift observed within roughly eight dialogue rounds, linked to declining attention toward the original system prompt.

Design implication: *Long-Conversation Stress Testing* — Evaluate characters using extended multi-turn conversations and measure where consistency begins to degrade.

Finding 2: Good Role-Play Requires Context-Dependent Reasoning

Du et al. (2026), HER — Surface-level imitation of speaking style is insufficient for strong role-playing. HER separates system-level planning and role-level internal reasoning, evaluating responses using context-dependent principles.

Design implication: *Context Builder + Context-Adaptive Evaluation* — Character behavior should depend on persona, retrieved knowledge, dialogue history, scene, relationship state, and current conversational situation.

Finding 3: Character Relationships Make Consistency More Consequential

Maples et al. (2024) — Users may perceive AI companions through relational roles such as friend, therapist, mirror, intelligence, or person.

Design implication: *Treat consistency as an interaction-quality and trust issue* — When a user views the character as a persistent social entity, contradictions or sudden behavioral changes may undermine the relationship itself.

Finding 4: Staying "In Character" Is Not Automatically Safe

Zhang et al. (2025) — Identifies harmful relational behaviors in AI companionship, including algorithmic compliance, where the AI may agree with or reinforce harmful user statements or behaviors.

Design implication: *Safety constraints must remain independent from persona* — A loyal, agreeable, aggressive, or emotionally attached character should not be allowed to override non-negotiable safety boundaries.

Central Synthesis — Component Trade-Offs

Prior work provides strong solutions for individual pieces. The open question is what happens when memory, retrieval, persona conditioning, context, and safety must operate together. Improving one component may create trade-offs elsewhere:

Stronger persona adherence
→ less situational flexibility?

More retrieval
→ persona-inappropriate knowledge?

More memory
→ knowledge boundary violations?

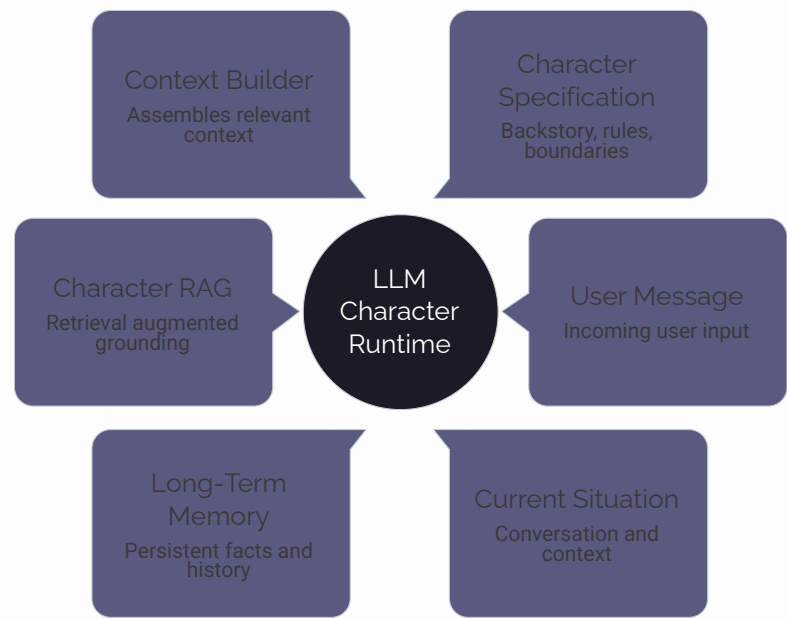
Stronger alignment
→ generic assistant behavior?

Li et al., 2024 · Maples et al., 2024 · Zhang et al., 2025 · Du et al., 2026

Character Consistency Workbench

Build the character as a system—not just a prompt.

System Architecture



Inputs

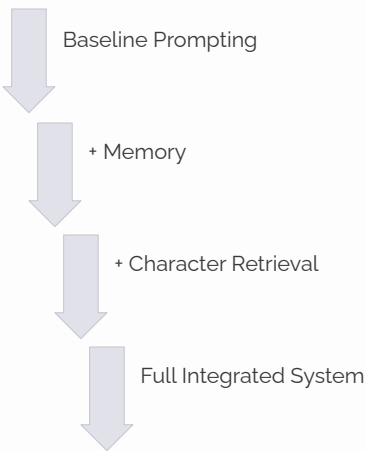
- **Character Specification**: Backstory, personality, relationships, knowledge boundaries, behavioral rules, dialogue style
- **User Message**
- **Current Situation**: Scene, relationship state, relevant world state, conversation state

Processing Modules

- Long-Term Memory**
Retrieve relevant information from prior interactions — promises, previous events, relationship history, user-specific facts
- Character RAG**
Retrieve character- and world-specific factual information — lore, world knowledge, backstory facts, known characters and locations
- Context Builder**
Combine character specification, memory, retrieved knowledge, dialogue history, scene, and relationship state into structured context for generation
- Persona & Style Conditioning**
Control speaking style, emotional profile, attitude, conversational habits, behavioral constraints

Experimental Comparison

Same Character. Same Scenario. Different System.



❑ The goal is not merely to combine components, but to **isolate what each component contributes** through controlled comparison.

Validate → Build → Evaluate

Checkpoint 1: Validate the Logic

Prompting Study — Test the same structured character specification and scenarios across ChatGPT, Claude, and Gemini using equivalent prompt chains.

Five scenario categories: Normal interaction · Long-context conversation · Adversarial persona challenge · Knowledge-boundary question · Context-dependent emotional or behavioral change

Main objective: Identify where prompt and context strategies fail before implementing the full system.



Checkpoint 2: Build the MVP

End-to-end prototype containing: Structured Character Specification · Long-Term Memory · Character RAG · Context Builder · Persona / Style Conditioning · Character Runtime · Evaluation & Debugging Layer

Desired demonstration: Define character → Run conversation → Detect failure → Inspect likely cause

Checkpoint 3: Evaluate

Controlled comparisons: Baseline Prompting vs. Integrated Character System, with ablations where feasible

Evaluate: Persona consistency · Memory consistency · Lore / factual grounding · Style stability · Situational appropriateness

Potential creator-facing user study: Can character creators more easily identify and repair problems using the workbench than through manual prompt rewriting?

Biggest Technical Unknown

Can stronger memory, retrieval, and persona conditioning improve long-term character coherence without making the character rigid, over-grounded, or contextually inappropriate?

Consistency ↔ Flexibility

Grounding ↔ Knowledge
Boundaries

Persona Adherence ↔ Situational Appropriateness

Closing Argument

Our goal is not simply to generate better character responses. It is to make character consistency something creators can systematically test, understand, and improve.

LLM characters are currently treated too much like prompts. This project treats them as **systems** that can be tested, inspected, and debugged — bringing engineering discipline to a problem that has so far been addressed through intuition and trial-and-error.